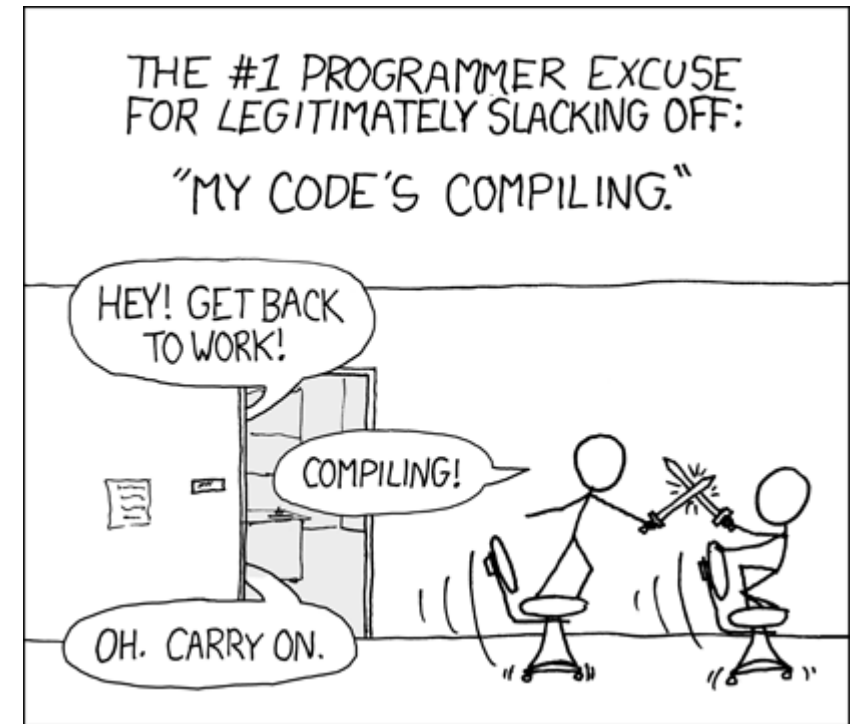




<https://imgs.xkcd.com/comics/compiling.png>

CMPT 295

Unit - Instruction Set Architecture

Lecture 27 – ISA Design – x295M - and Evaluation

Last Lecture

- Looked at an example of a RISC instruction set: MIPS
- MIPS follows ISA design guidelines
 1. Unambiguous **binary encoding** of machine code instruction
 2. MIPS functionally complete (“Turing complete”)
 3. Only 3 machine code instruction **formats**, but all have the **same length**
 - *R-format* for register
 - *I-format* for immediate
 - *J-format* for jump
 - Also, position of **fields** with **same purpose** are in the **same location** in all 3 formats

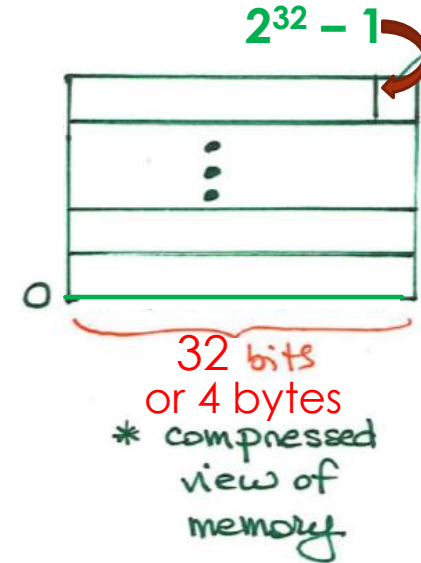
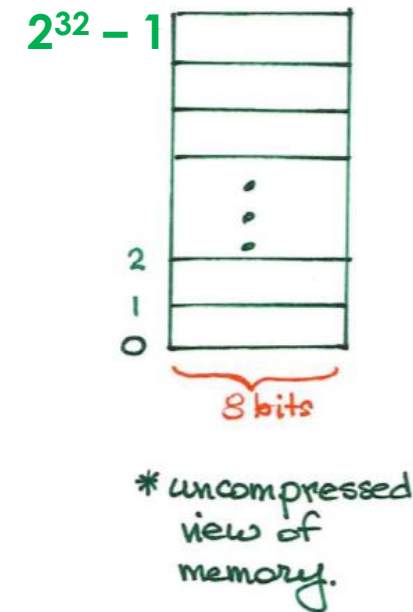
Today's Menu

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Example of an instruction set: MIPS
 - Create our own instruction sets
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Pipelined execution of machine instructions + Hazards

Let's design our own **ISA** - **x295M** (1 of 3)

1. Memory model

- **Word size** -> 32 bits
- **Size of memory** -> $2^m \times n$ bits
 - Size of a memory address (# of bits) is 32 bits, i.e., $m = 32$ bits
 - Range of memory addresses: 0 to $2^{32} - 1$
 - **Byte-addressable** memory model so, address resolution n is 8 bits or 1 byte
 - Size of memory -> $2^{32} \times 8$ bits or $2^{32} \times 1$ byte
- So, there are 2^{32} distinct addressable memory locations in this memory model and each of these addressable memory locations has 8 bits.



Let's design our own **ISA** - **x295M** (2 of 3)

2. Registers

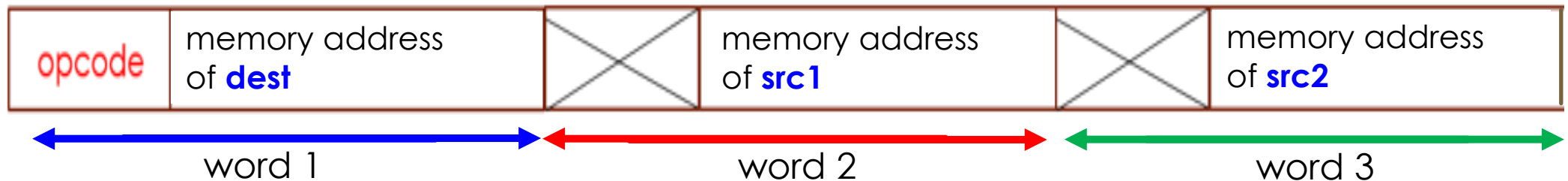
- **# of registers:** 0 registers (no general purpose registers)
- This model is called "**Memory Only**"
- **Data type:** 32 bits (like `int` in C)

Let's design our own ISA - **x295M** (3 of 3)

3. Instruction Set

x295M assembly language instructions	Semantic (i.e., Meaning)	Corresponding x295M machine code instructions
ADD a, b, c	$M[c] = M[a] + M[b]$	01 <30 bits> 00 <30 bits> 00 <30 bits>
SUB a, b, c	$M[c] = M[a] - M[b]$	10 <30 bits> 00 <30 bits> 00 <30 bits>
MUL a, b, c	$M[c] = M[a] * M[b]$	11 <30 bits> 00 <30 bits> 00 <30 bits>

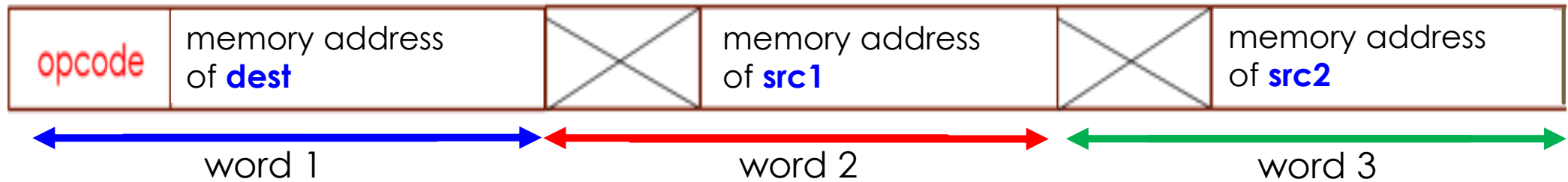
- **Memory addressing mode** -> **Direct** (**absolute** or **Imm** -> memory address)
- **Operand model** -> "3 Operand" model
- **Format** of corresponding **x295M** machine code instructions:



- Size of **opcode** -> **2 bits** Size of operand -> **30 bits**
- Length of 1 instruction -> _____ bits

Does **x295M** satisfy **ISA** design guidelines?

- It is not quite Turing complete
 - Which class(es) of instructions is/are missing?
- All the machine code instructions of **x295M** (corresponding to its assembly language instructions: **ADD**, **SUB** and **MUL**) can be expressing using this one format:



- Therefore, this leads to simple microprocessor design!

Instruction set (**IS**) design principles

1. Each instruction of **IS** must have an unambiguous **binary encoding** so microprocessor can unambiguously decode and execute it
2. **IS** must be functionally complete, i.e., must be “Turing complete”
 1. Data transfer instructions Storage reference
 2. Data manipulation instructions Arithmetic and logical
 3. Program control instructions Branch and jump} 3 classes of instructions
3. In terms of **machine code instruction format**:
 - a. **Ideally**, the machine code instructions in this **IS** must all have the **same length** and use the **same format**!
 - b. If we cannot achieve a., we must create as few formats as possible
 - c. If we cannot achieve a., then let's try to position the **fields** of our **machine code instruction formats** that have the same **purpose** at the **same location** in the **machine code instruction formats**

How good is **x295M**?

(1 of 6)

- Let's compare our **x295M** with **MIPS**
- To do so, we shall “**compile**” the following sample **C** code using both **IS**'s

sample **C** code: $z = (x + y) * (x - y)$

- The **x295M** assembly program is:

C code -> Assembly code

Meaning

ADD x, y, temp1
SUB x, y, temp2
MUL temp1, temp2, z

$M[c] = M[a] + M[b]$
 $M[c] = M[a] - M[b]$
 $M[c] = M[a] * M[b]$

ADD 0(\$sp), 4(\$sp), 12(\$sp)
SUB 0(\$sp), 4(\$sp), 16(\$sp)
MUL 12(\$sp), 16(\$sp), 8(\$sp)

$M[\$sp+12] = M[\$sp+0] + M[\$sp+4]$
 $M[\$sp+16] = M[\$sp+0] - M[\$sp+4]$
 $M[\$sp+8] = M[\$sp+12] * M[\$sp+16]$

Or, if our data is on the stack, our program becomes ...

Memory

Stack Segment

\$sp ->

How good is **x295M**?

(2 of 6)

- if **x** => **0x7ffffff00**, then our **x295M** assembly program becomes:

```
ADD 0x7ffffff00, 0x7ffffff04, 0x7ffffff0c
SUB 0x7ffffff00, 0x7ffffff04, 0x7ffffff10
MUL 0x7ffffff0c, 0x7ffffff10, 0x7ffffff08
```

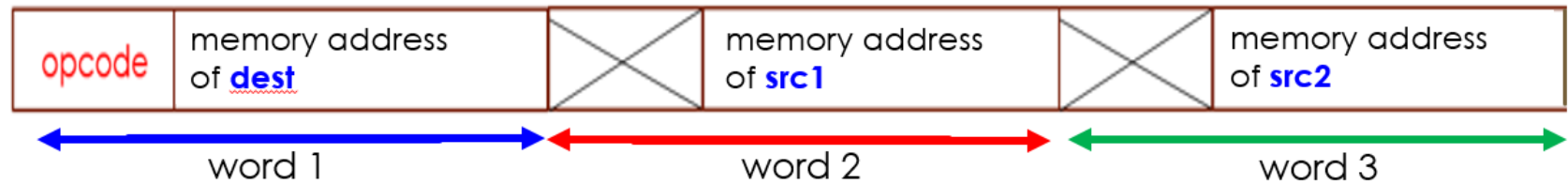
- Notice the pattern of the memory addresses:

in hexadecimal		in binary
0x7ffffff00	->	0111 1111 1111 1111 1111 1111 0000 0000
0x7ffffff04	->	0111 1111 1111 1111 1111 1111 0000 0100
0x7ffffff08	->	0111 1111 1111 1111 1111 1111 0000 1000
0x7ffffff0c	->	0111 1111 1111 1111 1111 1111 0000 1100
0x7ffffff10	->	0111 1111 1111 1111 1111 1111 0001 0000

How good is **x295M**?

(3 of 6)

- Now, let's "**assemble**" our **x295M** assembly program into a machine code program
- We shall do this using ...
- the format described in previous slide:



➤ Size of **opcode** -> 2 bits Size of operand -> 30 bits

- and this **opcode** encoding:

Opcode	Encoding
ADD	01
SUB	10
MUL	11
No op	00

How good is **x295M**?

(4 of 6)

- **Wait a minute?** The size of the operand, i.e., memory address is 30 bits. So, how do we shrink our memory addresses from 32 to 30 bits?
- **Answer** - Remember the pattern of the memory addresses we noticed:

0x7ffffff00	->	0111 1111 1111 1111 1111 1111 0000 0000	0x1ffffffc0
0x7ffffff04	->	0111 1111 1111 1111 1111 1111 0000 0100	0x1ffffffc1
0x7ffffff08	->	0111 1111 1111 1111 1111 1111 0000 1000	0x1ffffffc2
0x7ffffff0c	->	0111 1111 1111 1111 1111 1111 0000 1100	0x1ffffffc3
0x7ffffff10	->	0111 1111 1111 1111 1111 1111 0001 0000	0x1ffffffc4

- To shrink our memory addresses from 32 to 30 bits we shall drop the LS 2 bits which are always **00** anyway and we shall obtain the resulting memory addresses
- When the microprocessor decodes these machine code instructions, it will add these two **00** and create the correct memory addresses

How good is **x295M**? (5 of 6)

Assembly code instruction:

➡ So, our **x295M** machine code program is:

ADD 0x7ffffff00, 0x7ffffff04, 0x7ffffff0c

01 0x1ffffffc3 00 0x1ffffffc0 00 0x1ffffffc1

SUB 0x7ffffff00, 0x7ffffff04, 0x7ffffff10

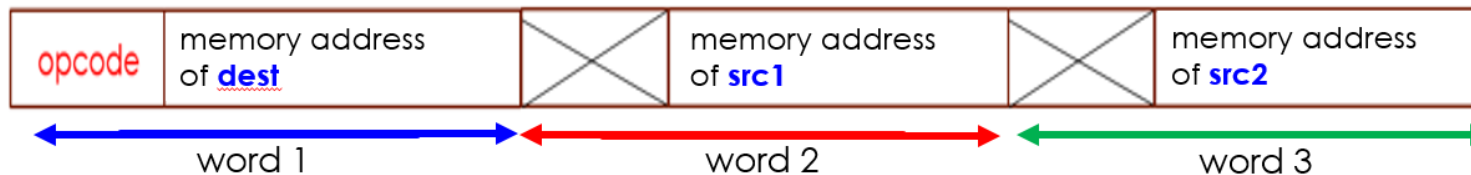
10 0x1ffffffc4 00 0x1ffffffc0 00 0x1ffffffc1

MUL 0x7ffffff0c, 0x7ffffff10, 0x7ffffff08

11 0x1ffffffc2 00 0x1ffffffc3 00 0x1ffffffc4

Machine code instruction:

Based on this format and opcode encoding:



➡ Size of **opcode** -> 2 bits Size of operand -> 30 bits

Opcode	Encoding
ADD	01
SUB	10
MUL	11
No op	00

How good is **x295M**? (6 of 6)

- When loaded (executed), each machine code instruction will be stored in the text segment of the memory

Memory address of
each instruction

Machine code

0x00400000	0x00400004	0x00400008
01 0x1fffffc3	00 0x1fffffc0	00 0x1fffffc1
0x0040000c	0x00400010	0x00400014
10 0x1fffffc4	00 0x1fffffc0	00 0x1fffffc1
0x00400018	0x0040001c	0x00400020
11 0x1fffffc2	00 0x1fffffc3	00 0x1fffffc4

Memory
Text Segment

0x00400000 ->

Now MIPS

(1 of 3)

➔ “**Compiling**” the sample **C** code

sample **C** code: $z = (x + y) * (x - y)$

using **MIPS** we get:

C code -> Assembly code

Meaning

```
lw  $s1, 0($sp)
lw  $s2, 4($sp)
add $s3, $s1, $s2
sub $s4, $s1, $s2
mul $s5, $s3, $s4
sw  $s5, 8($sp)
```

```
$s1 = M[$sp + 0]
$s2 = M[$sp + 4]
$s3 = $s1 + $s2
$s4 = $s1 - $s2
$s5 = $s3 * $s4
M[$sp + 8] = $s5
```

Memory
Stack Segment



Now **MIPS**

(2 of 3)

Opcode	Encoding
lw	35 ₁₀
sw	43 ₁₀

➔ Now, let's “**assemble**” our **MIPS** assembly program into a machine code program

I-format **src** **dest**

opcode	rs	rt	Address/immediate
6 bits	5 bits	5 bits	16 bits

Register	Number
\$s1	17 ₁₀
\$s2	18 ₁₀
\$s3	19 ₁₀
\$s4	20 ₁₀
\$s5	21 ₁₀
\$sp	29 ₁₀

Assembly code

Machine code

lw	\$s1, 0(\$sp) ->	100011	11101	10001	0x0000
lw	\$s2, 4(\$sp) ->	100011	11101	10010	0x0004
sw	\$s5, 8(\$sp) ->	101011	10101	11101	0x0008

Now MIPS

(3 of 3)

Opcode + func	Encoding
add	0 + 32 ₁₀
sub	0 + 34 ₁₀
mul	0 + 36 ₁₀

Register	Number
\$s1	17 ₁₀
\$s2	18 ₁₀
\$s3	19 ₁₀
\$s4	20 ₁₀
\$s5	21 ₁₀
\$sp	29 ₁₀

- Let's continue "assemble" our MIPS assembly program into a machine code program

R-format src1 src2 dest

opcode	rs	rt	rd	shamt	func
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Assembly code

add \$s3, \$s1, \$s2->

sub \$s4, \$s1, \$s2->

mul \$s5, \$s3, \$s4->

Machine code

000000	10001	10010	10011	00000	100000
000000	10001	10010	10100	00000	100010
000000	10011	10100	10101	00000	100100

Comparing our **x295M** with **MIPS**

Memory address of
each instruction

➡ Sample **C** code: $z = (x + y) * (x - y)$

0x00400000	01	01	1111	1111	1111	1111	1111	1100	0011
0x00400004	00	01	1111	1111	1111	1111	1111	1100	0000
0x00400008	00	01	1111	1111	1111	1111	1111	1100	0001
0x0040000c	10	01	1111	1111	1111	1111	1111	1100	0100
0x00400010	00	01	1111	1111	1111	1111	1111	1100	0000
0x00400014	00	01	1111	1111	1111	1111	1111	1100	0001
0x00400018	11	01	1111	1111	1111	1111	1111	1100	0010
0x0040001c	00	01	1111	1111	1111	1111	1111	1100	0011
0x00400020	00	01	1111	1111	1111	1111	1111	1100	0100

x295M
in machine code

Memory address of
each instruction

MIPS
in machine code

0x00400000	100011	11101	10001	0000	0000	0000	0000
0x00400004	100011	11101	10010	0000	0000	0000	0100
0x00400008	000000	10001	10010	10011	00000	100000	
0x0040000c	000000	10001	10010	10100	00000	100010	
0x00400010	000000	10011	10100	10101	00000	100100	
0x00400014	101011	10101	11101	0000	0000	0000	0008

Criteria used when comparing (evaluating) **ISAs**

1. Have the **instruction set (IS) design guidelines** been satisfied?
 1. Each instruction of **IS** have an unambiguous **binary encoding**
 2. **IS** is functionally complete -> i.e., it is “Turing complete”
 3. In terms of machine instruction format:
 - a. Create one format?
 - b. If a. is not possible, create as few formats as possible, all with the same length
 - c. If a. is not possible, position the **fields** that have the **same purpose** at the **same location** in all formats

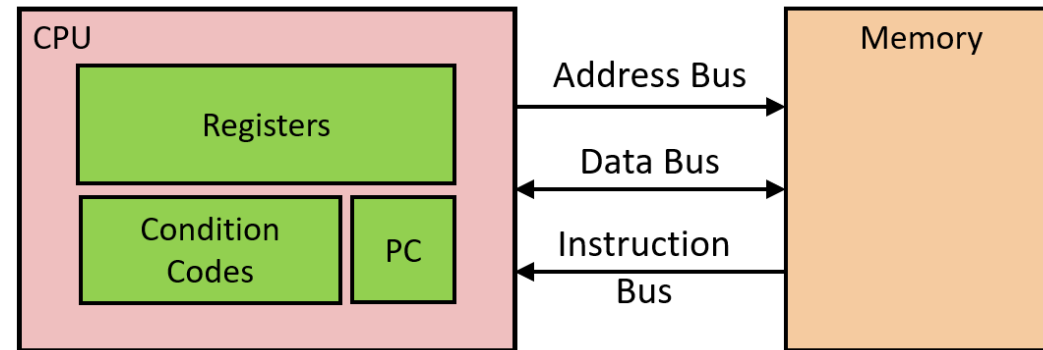
Criteria used when comparing (evaluating) **ISAs**

2. Program performance

- When an **ISA** design results in faster program execution then it is deemed “better”
- **Question:** What can affect the time a program takes to execute?
- **Answer:** Since accessing memory is slow (slower than accessing registers), the number of memory accesses a program does will affect its execution time
- Therefore, a possible criteria for evaluation is **number of memory accesses**
 - The fewer memory accesses our program makes, the faster it executes, hence the “better” it is

BTW, why is memory access slow?

- Memory access is the most time constraining aspect of program execution
- Why? Because of transfer rate limitation of the bus between memory and microprocessor
 - Memory is “far away” from the microprocessor so it takes time to transfer instructions and data from memory to microprocessor over the bus (wires)



- This is known as the **von Neumann bottleneck**
- From the above diagram, we can gather that register access is faster than memory access! Why?

von Neumann Bottleneck

Called
Memory Traffic

- It is created when **memory is accessed**
- When is memory accessed?
 - During the **fetch** stage of the microprocessor
 - When the microprocessor retrieves (fetches) an instruction from memory
 - During the **decode** and the **execute** stages of the microprocessor
 - When the microprocessor decodes and executes an instruction
 - During these stages, the microprocessor may **read the value of an operand from memory**
 - During these stages, the microprocessor may **write the result of the instruction to memory**

Decreasing the effect of **von Neumann bottleneck**

- We can decrease the effect of **von Neumann bottleneck** by reducing the **number of memory accesses**
 - **Memory Traffic**
- One way to achieve this is by introducing **registers** in the design of the **ISA**
 - Adding registers to the design of a microprocessor (i.e., microprocessor having access to registers) make machine code instructions shorter
- Reducing the number of operands required in machine code instructions (resulting in shorter instructions) may also help, but not always

Evaluation of our ISA **x295M** versus **MIPS** using **Memory Traffic** criteria

- Sample **C** code: $z = (x + y) * (x - y)$
- Let's count the **number of memory accesses**:

x295M

```
ADD 0($sp), 4($sp), 12($sp)
SUB 0($sp), 4($sp), 16($sp)
MUL 12($sp), 16($sp), 8($sp)
```

fetch decode/
execute

MIPS

fetch decode/
execute

```
lw  $s1, 0($sp)
lw  $s2, 4($sp)
add $s3, $s1, $s2
sub $s4, $s1, $s2
mul $s5, $s3, $s4
sw  $s5, 8($sp)
```

Total:

Summary

- ✓ We created our own **instruction set (IS)**
 - ✓ **x295M**: “Memory only”
- ✓ We checked whether or not **x295M** followed **ISA** design guidelines?
- ✓ How good was our **x295M**? -> Evaluation of **ISA**
 - ✓ We compared **x295M** with **MIPS**
 - ✓ Compiled a sample **C** program (code) using both **ISAs**
 - ✓ Then evaluated both **ISAs**
- ✓ When we evaluate **ISA** we use the **Memory Traffic** criteria:
 - ✓ This means that we examine the effect of the **von Neumann bottleneck** on the execution time of our program by counting **number of memory accesses**
 - ✓ The fewer memory accesses our program makes, the faster it executes, hence the “better” it is
 - ✓ Hence, reducing the **number of memory accesses** decreases the effect of **von Neumann bottleneck** on our program execution

Next Lecture

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Look at an example of an instruction set: MIPS
 - Create our own
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Pipelined execution of machine instructions + Hazards